

Prague University of Economics and Business

Faculty of Informatics and Statistics



USING SYNTHETIC TEXT DATA TO TRAIN SENTIMENT ANALYSIS MODELS

Author: Ilia Frolov

Prague, May 2025

Abstract

The explosive growth of digital text data presents many opportunities and challenges, especially in the area of sentiment analysis. A common problem is the imbalance of datasets, where one sentiment class significantly outperforms others, resulting in prediction bias. In this study, the class imbalance problem is addressed by creating synthetic data using GPT-2, a state-of-the-art generative language model. Comprehensive experiments were conducted using logistic regression, long-term short-term memory (LSTM), and bidirectional encoding representations from transformers (BERT) models. The evaluation showed significant results, in particular BERT, which achieved accuracies up to 74.2% when trained solely on synthetic data. This highlights the value of using synthetic data in NLP applications. The study suggests opportunities for future research using more advanced language models and broader NLP applications.

Keywords

Synthetic data, GPT-2, Sentiment analysis, Logistic Regression, LSTM, BERT, Class imbalance, Data augmentation, NLP.

Content

Introduction.....	6
1.1 Goals and Research Questions.....	6
1.2 Literature Review	6
1.2.1 Synthetic Data Generation Techniques	6
1.2.2 Metrics and Evaluation Approaches	7
1.3 Methodology	7
1.4 Data Source.....	7
1.4.1 Data Utilization	8
1.4.2 Dataset Preview	8
2 Theoretical Background.....	11
2.1 NLP	11
2.2 Data Augmentation.....	12
2.2.1 The main techniques used in text data augmentation	13
2.2.2 Evaluation and Selection of Augmentation Methods	13
2.2.3 Application in the project	14
2.2.4 Conclusion	15
3 Models used.....	16
3.1 Logistic regression.....	16
3.1.1 Overview	16
3.1.2 Textual Data Transformation	16
3.1.3 TF-IDF Vectorization	17
3.1.4 Model Training	17
3.1.5 Interpretation and Application.....	17
3.2 BERT.....	18
3.2.1 General Description.....	18
3.2.2 Application in This Thesis	18
3.3 LSTM.....	19
3.3.1 General Overview.....	19
3.3.2 Model Architecture	19
3.3.3 Application in This thesis	20
4 Experiments	21
4.1 Code description.....	21
4.1.1 Environment Setup and Library Installation	21

4.1.2	Google Drive Integration for Data Handling.....	22
4.1.3	TPU Initialization.....	22
4.1.4	Real Data Loading.....	23
4.1.5	Synthetic Data Generation Using GPT-2.....	23
4.1.6	Synthetic Data Quality Evaluation.....	24
4.1.6.1	Perplexity.....	24
4.1.6.2	Word Map.....	25
4.1.7	Data Combination and Class Balancing.....	27
4.1.8	Logistic Regression Modeling	27
4.1.9	LSTM Model and Preprocessing	27
4.1.10	BERT-Based Transformer Model Experiments	28
4.1.11	Evaluation Utilities	29
4.2	Results.....	29
4.2.1	Tables	29
4.2.2	Discussion.....	33
4.2.3	Interpretation	34
4.2.5	Conclusions	36
	List of references	38

List of abbreviations

NLP – Natural Language Processing

GPT – Generative Pre-trained Transformer

GAN – Generative Adversarial Network

LSTM – Long Short-Term Memory

BERT – Bidirectional Encoder Representations from Transformers

F1-Score – Harmonic Mean of Precision and Recall

TPU – Tensor Processing Unit

CPU – Central Processing Unit

GPU – Graphics Processing Unit

Introduction

Natural Language Processing (NLP) is increasingly gaining key importance in the digital age, impacting areas ranging from business intelligence to customer sentiment assessment. One of the most important applications of NLP is sentiment analysis, which faces challenges such as class imbalances that make machine learning models ineffective or give biased predictions. To counter this, recent advances have been made in generating synthetic data, particularly through powerful language models such as GPT-2. The broader implications of generative AI in industry and research are explored in Harvard Business Review's recent overview (Mollick et al., 2024). This study investigates the effectiveness of synthetic text data for improving sentiment analysis models, testing the hypothesis that synthetic data can successfully supplement or replace real data.

1.1 Goals and Research Questions

The goal of the study is: to test different language models on real and synthetic data at different ratios

The Research questions are:

1. Is it possible to replace real data with synthetic data when training a model without loss in accuracy?
2. If replacement is possible, what percentage of the data can be replaced?

1.2 Literature Review

This section provides an overview of existing research related to synthetic data generation and evaluation in natural language processing. It highlights key techniques, relevant studies, and commonly used performance metrics that form the foundation for the present work.

1.2.1 Synthetic Data Generation Techniques

Earlier work utilizing GPT-2 demonstrated the feasibility of generating synthetic text to augment training datasets (Whitfield, 2020). In parallel, studies have explored the use of data augmentation techniques in NLP (Shorten et al., 2021) and have highlighted the potential of synthetic data to address class imbalance. Additionally, approaches based on adversarial networks (Imran et al., 2022) have been proposed for enhancing the diversity of generated text, further contributing to

the potential of synthetic data to augment NLP training datasets (Suhaeni and Yong, 2023; 2024).

Recent work has also emphasized the importance of evaluating the quality of synthetic text. For instance, Gupta (2023) examined the BERTScore of GAN-generated text in the context of sentiment analysis, highlighting that while synthetic data can closely resemble human-written content in form, it varies in semantic precision and downstream performance. Similarly, Imran et al. (2022) demonstrated that synthetic data generated through GANs can significantly improve sentiment classification tasks, especially in scenarios with limited labeled data, underlining its practical utility.

1.2.2 Metrics and Evaluation Approaches

Evaluating the impact of synthetic data on model performance requires a set of reliable and interpretable classification metrics. Traditional measures such as accuracy and F1-score remain central in this context, offering a general understanding of how well a model distinguishes between classes (Šimůnek, 2024). However, as Gupta (2023) notes, these metrics alone may be insufficient in imbalanced settings. Precision and recall become particularly relevant here: precision reflects the proportion of correctly predicted positive instances among all predicted positives, while recall emphasizes the model's ability to identify all actual positive instances. Used together, these four metrics provide a more complete picture of model effectiveness, especially when synthetic data is used to address class imbalance. This is consistent with Šimůnek's (2024) work, where such metrics were key in evaluating classifier performance on imbalanced datasets with various balancing techniques

1.3 Methodology

This research employs a controlled experimental approach using GPT-2-generated synthetic data to augment real datasets. Sentiment analysis performed using Logistic Regression, LSTM, and BERT across five real-to-synthetic data proportions: 0%, 25%, 50%, 75%, and 100%. Key metrics include accuracy, precision, recall, F1-score, and ROC-AUC.

1.4 Data Source

The dataset titled "IMDB Movie Reviews Dataset", created and maintained by StanfordAI, was downloaded from Kaggle on November 20, 2024. This dataset contains 50,000 movie reviews categorized as 25000 positive and 25000 negative,

and it is widely used for sentiment analysis benchmarking. The dataset can be accessed at Kaggle

1.4.1 Data Utilization

To manage computational constraints, a subset of 5000 reviews was extracted from the dataset for training and validation purposes. An additional subset consisting of 1000 reviews was reserved exclusively for testing to assess model performance objectively.

This structured approach ensures efficient computational processing while maintaining sufficient data volume for robust model training and evaluation.

1.4.2 Dataset Preview

Below is a brief preview of the data entries from the dataset:

Review	Sentiment
"One of the other reviewers has mentioned that..."	positive
"A wonderful little production. The filming technique is very unassuming..."	positive
"I thought this was a wonderful way to spend time on a too hot summer weekend..."	positive
"Basically there's a family where a little boy thinks there's a monster in his closet..."	negative
"Petter Mattei's 'Love in the Time of Money' is a visually stunning..."	positive

Word map for positive reviews: 1



Figure 1 source: author

Word map for negative reviews: 1



Figure 2 source: author

Considering that World maps can be difficult to analyze, tables of the most common words in the dataset are attached below.

Positive reviews		Negative reviews	
Film	4069	Movie	4685

Movie	3672	Film	3636
One	2635	One	2468
Like	1773	Like	2147
Good	1465	Even	1502
Great	1275	Good	1478
Story	1231	Bad	1397
See	1202	Would	1363
Time	1195	Really	1277
well	1103	time	1159

2 Theoretical Background

The topic of this research lies in NLP and Sentiment Analysis and the models used.

2.1 NLP

Natural language processing (NLP) is a subfield of artificial intelligence (AI) that aims to facilitate computers' comprehension, interpretation, and generation of human language in a manner that is both meaningful and useful. It draws upon techniques from the fields of linguistics, computer science, and machine learning to facilitate the analysis and processing of text or speech data. (Shorten et al., 2021).

NLP applications encompass a range of tasks, from relatively simple processes such as text classification and sentiment analysis to more complex operations such as machine translation, question answering, and the development of conversational agents (chatbots). At its core, NLP seeks to establish a bridge between human communication and machine understanding. This enables systems to extract insights from unstructured data, such as emails, tweets, or customer reviews. (Gupta, 2023)

The processing steps in NLP include:

- **Tokenization:** The initial preprocessing step involves dividing text into smaller, manageable units known as tokens, such as words, phrases, or sentences. This facilitates subsequent analysis by allowing the text to be quantitatively represented and analyzed.
- **Normalization:** Following tokenization, normalization involves standardizing text data to a consistent format, including converting text to lowercase, removing punctuation, correcting spelling errors, and handling special characters or accents. This step ensures uniformity across the dataset, making analyses more accurate.
- **Stop Word Removal:** Stop words are frequently occurring words (e.g., "the", "is", "and") that often carry minimal semantic value. Removing stop words reduces noise and computational complexity, highlighting the more meaningful words.
- **Stemming and Lemmatization:** Stemming reduces words to their root form by removing suffixes and prefixes, often using heuristic rules. Lemmatization, however, reduces words to their dictionary or canonical

form, considering their grammatical context, thus preserving meaning more effectively than stemming.

- **Part-of-Speech Tagging:** This method assigns grammatical labels to tokens, such as nouns, verbs, adjectives, and adverbs. These tags provide syntactic context and can enhance subsequent analytical processes like parsing or entity recognition.
- **Named Entity Recognition (NER):** NER identifies and classifies key entities within text, such as people's names, organizations, locations, dates, and numerical values. This is crucial for structuring data and extracting meaningful insights.

The methods and outputs of NLP include:

- **Sentiment Analysis:** This technique assesses the sentiment or emotional tone within text data, categorizing it as positive, negative, or neutral. It is widely applied in consumer feedback analysis, market research, and social media monitoring (Gupta, 2023; Imran et al., 2022).
- **Machine Translation:** This method converts text from one language to another automatically, facilitating cross-lingual communication and content accessibility globally.
- **Speech Recognition:** This technology transcribes spoken language into written text, forming the basis for voice-activated systems and speech-enabled interfaces.
- **Text Classification:** Categorizing text documents into predefined classes based on their content. Common applications include spam detection, topic labeling, and content organization.
- **Topic Modeling:** Identifying abstract topics present in a collection of documents by detecting patterns and groupings of words. Techniques like Latent Dirichlet Allocation (LDA) are commonly used.

Natural language processing (NLP) is a pervasive technology employed in a multitude of applications, including virtual assistants (e.g., Siri, Alexa), automated customer support, recommendation systems, and content moderation. As a result, NLP has become an indispensable tool in the realm of modern technology.

2.2 Data Augmentation

Text data augmentation is a widely employed technique in natural language processing (NLP) aimed at artificially expanding and diversifying datasets. The

primary objective is to enhance the generalizability and robustness of machine learning models by generating varied but semantically consistent versions of existing data. This process is essential for improving model performance, mitigating overfitting, and addressing data imbalance issues (Shorten et al., 2021; Imran et al., 2022).

2.2.1 The main techniques used in text data augmentation

- **Synonym Replacement:** This technique involves replacing words or phrases with their synonyms to introduce minor variations while keeping the original meaning intact. For example, the sentence "The movie was amazing" could be transformed into "The film was incredible."
- **Random Insertion and Deletion:** Randomly adding or removing words from a sentence while preserving its grammatical structure helps create variability. For instance, "This is a superior product" could be modified to "This product is of superior quality."
- **Back Translation:** Translating text into another language and then translating it back to the original language introduces natural variations. For example, an English sentence might be translated into French and then back into English, slightly altering the phrasing.
- **Paraphrasing:** Using NLP models, such as GPT or T5, sentences or passages can be rephrased to convey the same meaning in different words. For instance, "The weather is nice today" might become "The current meteorological conditions are agreeable."
- **Contextual Word Embedding Augmentation:** Models like BERT are used to substitute or insert words based on their semantic context, enhancing the diversity of the dataset. For example, "The food was delicious and flavorful" could become "The meal was exquisite and tasty."
- **Random Noise Injection:** Introducing minor typographical errors or inconsistencies simulates natural variability in text. For example, "I am passionate about programming" could be slightly altered to "I am enthusiastic about programming."

2.2.2 Evaluation and Selection of Augmentation Methods

The effectiveness of augmentation techniques can vary based on application context, dataset characteristics, and model architecture. Proper evaluation involves:

- **Linguistic coherence:** Ensuring augmented sentences remain grammatically correct and semantically meaningful.

- **Impact on model performance:** Empirically measuring improvements through validation metrics such as accuracy, precision, recall, and F1-score.

Advanced evaluation metrics like perplexity scores or semantic similarity measures (e.g., BERTScore) can be employed to quantify the quality of augmented data. (Gupta, 2023)

Applications of Text Data Augmentation

Text data augmentation proves particularly beneficial in various NLP domains:

- **Sentiment Analysis:** Enhancing model ability to capture subtle emotional nuances by diversifying sentiment expressions.
- **Text Classification:** Improving robustness against varying textual expressions within different categories.
- **Machine Translation:** Providing diversified training samples to strengthen translation capabilities across linguistic variations.

Challenges and Limitations

Despite significant benefits, augmentation methods also introduce challenges:

- Risk of altering original meanings or introducing noise that negatively impacts model learning.
- Difficulty in assessing semantic equivalence and maintaining dataset coherence across large-scale augmentation.

Careful method selection, rigorous quality control, and empirical validation are crucial to maximizing augmentation efficacy.

2.2.3 Application in the project

This study deliberately focuses on a generative approach to data augmentation using synthetic text. Instead of modifying existing data samples, new reviews are generated from scratch using a pretrained language model (GPT-2). This decision is motivated by the need to explore the frontier of generative AI's capacity to simulate labeled data and address class imbalance in a more scalable and flexible way. Generative models create samples that are not bound by the structure or limitations of the original dataset.

2.2.4 Conclusion

Text data augmentation is particularly beneficial when datasets are limited, helping machine learning models generalize better. By diversifying the training data, models are less likely to overfit specific patterns, thereby improving their performance on new, unseen examples. This approach is commonly applied in areas such as sentiment analysis, text classification, and machine translation (Gupta, 2023; Imran et al., 2022).

3 Models used

The models selected for this study - logistic regression, LSTM, and BERT were chosen to represent a spectrum of complexity in natural language processing approaches, ranging from traditional statistical methods to modern deep learning architectures. Logistic regression serves as a simple and interpretable baseline, allowing for transparent evaluation of how much performance gain is achieved through more advanced methods. LSTM, as a classic sequence model, captures temporal dependencies in text and reflects earlier deep learning advancements in NLP. Finally, BERT represents the state of the art in contextual language understanding, offering high accuracy and robustness through its transformer-based architecture. This diverse model selection enables a meaningful comparison of how synthetic data impacts performance across different levels of model sophistication.

3.1 Logistic regression

This section presents logistic regression as a baseline model for sentiment classification. It outlines its theoretical basis, describes how textual data is transformed for model input, and explains the training and interpretation process in the context of this thesis.

3.1.1 Overview

Logistic regression is a statistical model widely used for binary classification tasks. It is particularly suitable for scenarios that involve predicting categorical outcomes with two possible classes, such as determining whether a review sentiment is positive or negative. Logistic regression estimates the probability that a given instance belongs to a particular class by applying a logistic sigmoid function, transforming inputs into a value between 0 and 1.

3.1.2 Textual Data Transformation

For logistic regression to process textual data, the data must first be transformed into numerical features. A common and effective method used for this transformation is Term Frequency-Inverse Document Frequency (TF-IDF). The TF-IDF technique assigns weights to words based on their frequency in individual documents relative to their frequency across all documents, highlighting terms that are distinctive and important within a specific context.

3.1.3 TF-IDF Vectorization

The `TfidfVectorizer` tool from `scikit-learn` converts textual reviews into numerical feature vectors. Specifically, it calculates TF-IDF scores for each word, generating a sparse numerical matrix where each row represents a review and each column represents a unique term from the corpus. The values in this matrix represent the TF-IDF weights, indicating the importance of each term to a particular review.

TF-IDF was selected due to its simplicity and interpretability. Unlike dense word embeddings, TF-IDF does not require large-scale pretraining and provides transparent, sparse feature representations that align well with the linear assumptions of logistic regression. Its ability to emphasize distinctive terms helps improve classification performance, particularly in settings with limited computational resources or smaller datasets.

Here is an illustrative example of TF-IDF vectorization for the set of reviews looks like:

and	br	in	is	it	of	the	this
0.241889	0.241889	0.142996	0.362834	0.339967	0.282204	0.645039	0.142996
0.347499	0.297856	0.000000	0.148928	0.209313	0.248214	0.794284	0.000000
0.273938	0.273938	0.323882	0.273938	0.192505	0.273938	0.547875	0.404853
0.363652	0.545478	0.214977	0.181826	0.000000	0.181826	0.545478	0.107488

3.1.4 Model Training

Once text data has been converted into numerical features using TF-IDF, the logistic regression model is trained by optimizing its coefficients. During training, the model adjusts these coefficients to minimize the cross-entropy loss, effectively finding the optimal weights for predicting sentiment accurately. The trained model can subsequently classify new reviews by estimating their probabilities of belonging to the positive or negative sentiment class.

3.1.5 Interpretation and Application

Logistic regression's simplicity and interpretability make it a suitable choice for sentiment analysis, providing clear insights into which terms significantly influence review sentiment. Its computational efficiency and straightforward implementation further enhance its practicality in large-scale sentiment classification tasks.

3.2 BERT

This section introduces BERT, a transformer-based model that represents the state of the art in NLP. It explains the model's architecture, pretraining approach, and details how BERT was fine-tuned and applied to sentiment analysis within this thesis.

3.2.1 General Description

BERT (Bidirectional Encoder Representations from Transformers) is a deep learning model developed by Google in 2018 that significantly advanced the field of natural language processing (NLP). Unlike traditional language models that process text in a unidirectional manner (either left-to-right or right-to-left), BERT reads text bidirectionally. This enables it to better capture the context of words based on both their preceding and following tokens (Devlin et al., 2018).

The model architecture is based on the Transformer encoder, introduced by Vaswani et al. (2017). It utilizes attention mechanisms to weigh the importance of different words in a sequence, allowing BERT to understand complex linguistic patterns such as polysemy and context-dependent meanings.

BERT is pre-trained on large corpora, including the English Wikipedia and BookCorpus, through two unsupervised tasks:

- **Masked Language Modeling (MLM):** Random words in a sentence are masked, and the model is trained to predict them based on surrounding context.
- **Next Sentence Prediction (NSP):** The model is trained to determine whether one sentence logically follows another.

Thanks to this pretraining strategy, BERT can be fine-tuned on specific downstream tasks, such as sentiment analysis, named entity recognition, or question answering, with relatively small amounts of task-specific data.

BERT exists in various sizes. In this project, the "bert-base-uncased" model was used, which consists of 12 layers (transformer blocks), 768 hidden units, and 12 attention heads, totaling approximately 110 million parameters.

3.2.2 Application in This Thesis

In the experimental part of this thesis, BERT was used for binary sentiment classification (positive vs. negative). A pre-trained model (bert-base-uncased) was loaded using the Hugging Face Transformers library. The main steps involved were:

- **Tokenization:** Texts were tokenized using the BertTokenizer, truncated or padded to a maximum sequence length of 100 tokens (see function `prepare_dataset` in the code).
- **Fine-tuning:** In this thesis, the pretrained model `TFBertForSequenceClassification` was loaded and trained end-to-end for binary sentiment classification. This includes updating both the classification head and the underlying BERT encoder weights. The model was optimized using the Adam optimizer (learning rate 2×10^{-5}), with early stopping based on validation loss. This setup follows the standard fine-tuning procedure for transformer-based models and was implemented using the Hugging Face Transformers library in TensorFlow.
- **Evaluation:** After training, the model's performance was evaluated using accuracy, precision, recall, F1-score, and AUC-ROC (see function `evaluate_model`).

Experiments were conducted across different proportions of real and synthetic data (0%, 25%, 50%, 75%, and 100% real data), and the results demonstrated that BERT consistently outperformed other models in sentiment classification tasks.

3.3 LSTM

This section focuses on the Long Short-Term Memory (LSTM) model, a type of recurrent neural network designed to capture sequential patterns in text. It outlines the model's architecture and describes its implementation and application in the context of binary sentiment classification.

3.3.1 General Overview

Long Short-Term Memory (LSTM) networks are a special kind of recurrent neural network (RNN) designed to capture long-range dependencies in sequential data. Whereas a standard RNN can suffer from vanishing or exploding gradients when learning relationships over many time steps, LSTM units incorporate internal gating mechanisms (input, forget, and output gates) that regulate the flow of information through the cell. This allows an LSTM to selectively remember or discard information over arbitrarily long sequences, making it particularly well suited to tasks such as language modeling, speech recognition, and sentiment analysis (Imran et al., 2022; Gupta, 2023).

3.3.2 Model Architecture

The typical LSTM-based sentiment classifier consists of the following components:

Embedding Layer:

Converts each token in the input sequence into a dense, real-valued vector of fixed dimensionality. These embeddings may be learned from scratch or initialized from a pre-trained source (Shorten et al., 2021).

Bidirectional LSTM Layer:

Processes the sequence in both forward and backward directions. Forward LSTM captures dependencies from the beginning of the sequence to the end, while backward LSTM captures context in reverse. Their outputs are concatenated, enabling the model to leverage information from both past and future tokens simultaneously (Gupta, 2023).

Dense Output Layer:

A fully connected layer with a single sigmoid-activated unit, which outputs a probability score for the positive class (with its complement representing the negative class) (Šimůnek, 2024).

3.3.3 Application in This thesis

In the experimental part of this thesis, an LSTM-based model was implemented to perform binary sentiment classification, determining whether a given movie review is positive or negative. The model was developed using the TensorFlow Keras API.

Before training, all text data was preprocessed by tokenizing the reviews into individual words and converting them into numerical sequences using a tokenizer. Each sequence was then padded to a uniform length of 100 tokens to ensure consistent input shape for the model.

The model was compiled using the **Adam optimizer**, and trained with **early stopping** based on validation loss to prevent overfitting.

Experiments were conducted across five different proportions of real-to-synthetic data: 0%, 25%, 50%, 75%, and 100% real data. This allowed for a comprehensive evaluation of the LSTM model's ability to learn from synthetic text generated by GPT-2, and to determine how performance changes when real data is partially or fully replaced.

4 Experiments

This chapter describes the experimental framework used to evaluate the effectiveness of synthetic data in training sentiment analysis models. It outlines the technical setup, data preparation, model training procedures, and presents the results and insights derived from multiple experimental scenarios.

The scheme of the experiment 1

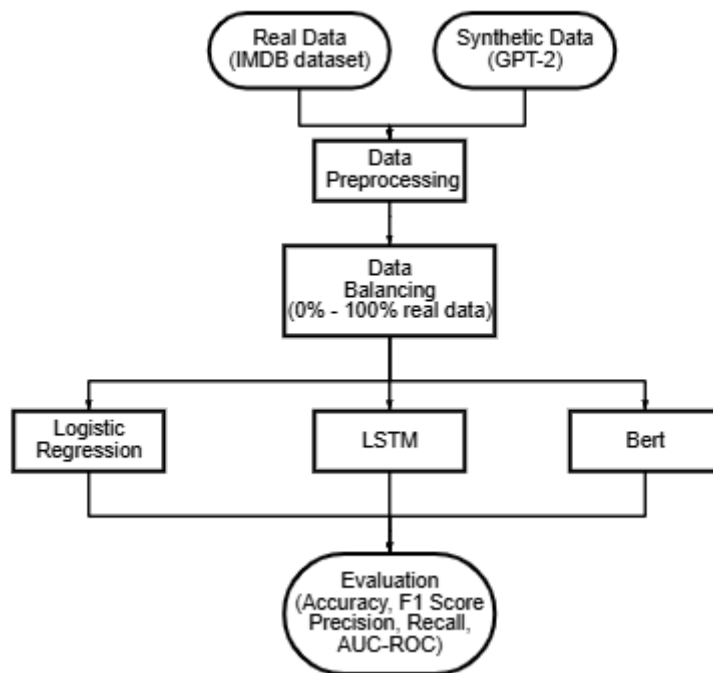


Figure 3 source: author

4.1 Code description

The following part contains the description of the experiment. The full implementation of the experimental pipeline is available in the accompanying GitHub repository [Frolov, 2025].

4.1.1 Environment Setup and Library Installation

The initial section sets up the computational environment by installing and importing essential Python libraries. These include TensorFlow and Transformers for deep learning and NLP, Pandas and NumPy for data operations, Scikit-Learn for classical ML and evaluation metrics, and Matplotlib/Seaborn for visualization. Gdown is used for file retrieval from Google Drive, and WordCloud aids in

visualizing textual distributions. These libraries support various aspects of data processing, deep learning model construction, visualization, and statistical evaluation necessary for NLP tasks.

```
!pip install tensorflow==2.15.0
!pip install transformers==4.38.2
!pip install pandas==1.5.3
!pip install numpy==1.26.4
!pip install scikit-learn==1.4.2
!pip install matplotlib==3.7.1
!pip install seaborn==0.12.2
!pip install gdown==5.1.0
!pip install wordcloud
```

4.1.2 Google Drive Integration for Data Handling

The code configures Google Drive integration, establishing file paths for storage and retrieval of synthetic datasets. This integration facilitates easy access to data and seamless execution within cloud environments such as Google Colab

```
from google.colab import drive
drive.mount('/content/drive')
```

4.1.3 TPU Initialization

To enhance training efficiency, especially for large-scale models like BERT and GPT-2, the code includes logic for TPU detection and configuration. The code dynamically initializes Tensor Processing Units (TPU) if available, significantly accelerating deep learning computations. If a TPU is not detected, it gracefully defaults to available CPUs or GPUs, ensuring versatile deployment across diverse computational resources.

```
tpu = tf.distribute.cluster_resolver.TPUClusterResolver()
strategy = tf.distribute.TPUStrategy(tpu)
```

4.1.4 Real Data Loading

The `load_real_data_from_drive` function retrieves the IMDB dataset comprising labeled movie reviews. Reviews labeled 'positive' or 'negative' are mapped into numerical binary classes (1 for positive, 0 for negative), preparing the dataset for machine learning training and evaluation. It utilizes the IMDB dataset.

```
real_data = pd.read_csv(output_path)

real_data['label'] = real_data['sentiment'].map({'positive': 1, 'negative': 0})
```

4.1.5 Synthetic Data Generation Using GPT-2

Synthetic dataset is generated using GPT-2. It was selected for synthetic data generation due to its open availability, ease of fine-tuning, and lower computational requirements compared to newer models such as GPT-3 or GPT-4. While more recent models offer improved performance, the transparency and accessibility of GPT-2 make it a practical choice for controlled academic experimentation.

One of the central functions in the code is the dynamic generation of prompts for creating synthetic data. This is done by using the `generate_dynamic_prompt(sentiment)` function, which constructs context-rich prompts tailored to the desired sentiment label either 'positive' or 'negative'.

The approach ensures variability in the synthetic movie reviews generated by the GPT-2 model. Instead of using a static template, the function randomly selects from predefined sets of adjectives and reasons to compose natural and emotionally expressive prompts.

For example, for a positive sentiment, the function might generate a prompt such as:

```
"Write an enthusiastic and detailed movie review. The movie was fantastic because the plot kept me on the edge of my seat from start to finish."
```

For a negative sentiment, a corresponding prompt might be:

```
"Write a critical and detailed movie review. The movie was dull because the visual effects looked unconvincing."
```

These prompts are passed to the GPT-2 model using the `generate_synthetic_data_batch()` function, which tokenizes the inputs and generates text sequences beyond the prompt length. This design ensures that the model's output is driven by structured input but remains diverse and creative.

Importantly, the dynamic nature of the prompts lowers the risk of producing repetitive outputs, improving the generalization quality of the synthetic dataset.

This is particularly valuable for training robust sentiment classifiers that should perform well on nuanced and varied text samples.

The code defines a mechanism to generate labeled synthetic data using the GPT-2 model. Dynamic prompt creation is handled via:

```
def generate_dynamic_prompt(sentiment):
```

Prompts are tailored based on the target sentiment, ensuring diversity and realism in generated content. The GPT-2 model is invoked with:

```
model.generate(..., max_length=..., temperature=0.7, top_k=50, top_p=0.95, ...)
```

where:

- **max_length** is the total number of tokens (including those in the prompt) that the model is allowed to produce.
- **Temperature** is a scalar value $T > 0$ that smooths the model's predicted probability distribution before sampling. Lower temperatures ($T < 1$) makes the distribution peakier (more conservative, high-probability tokens are favored), while higher temperatures ($T > 1$) flattens the distribution (increasing randomness). In our case, `temperature=0.7` biases toward more likely continuations while still allowing variety.
- **top_k** limits sampling to the k tokens with the highest probabilities at each generation step. All other tokens are masked out (zero probability).
- **top_p** dynamically selects the smallest set of tokens whose cumulative probability mass exceeds p . For `top_p=0.95`, at each step one first sorts tokens by descending probability and then takes the minimal prefix whose probabilities sum to at least 0.95.

Outputs are post-processed by `clean_generated_text()` to eliminate prompt remnants and refine readability. The final outputs are stored in CSV format for subsequent training use.

4.1.6 Synthetic Data Quality Evaluation

This section evaluates the quality of the synthetic text generated by GPT-2. It introduces key assessment methods, including perplexity scores and visualizations such as word maps, to determine the fluency, coherence, and usefulness of the generated data for sentiment analysis tasks.

4.1.6.1 Perplexity

To assess quality, the code computes the perplexity of the synthetic texts:

```
def compute_perplexity(model, tokenizer, texts):  
    ...  
    perplexity = np.exp(avg_loss)
```

Perplexity measures how well the language model predicts each word in the dataset. Lower perplexity values indicate more fluent and coherent outputs, suggesting that the synthetic reviews closely resemble natural language.

The average perplexity of the synthetic text generated by GPT-2 was 13.99, indicating a high degree of fluency and consistency. This supports the hypothesis that well-structured synthetic data can effectively replace real data in sentiment classification tasks.

To place our result (average PPL = 13.99) in context:

- **GPT-2 on WikiText-2:**
The original GPT-2 paper reports PPL ≈ 19.93 on WikiText-2 under a no-overlap evaluation strategy (Radford et al. 2019).
- **GPT-2 on Penn Treebank:**
GPT-2 achieves PPL ≈ 35.76 on the Penn Treebank corpus, reflecting the greater variability and complexity of that dataset (Radford et al. 2019).

Our synthetic reviews generated by the same GPT-2 architecture got a perplexity of 13.99. This value is lower than typical benchmark scores for human-written text, indicating that the synthetic reviews are highly predictable.

However, one must interpret a very low perplexity with care: it can also suggest that the text closely follows the model's learned patterns. In our case, a PPL of 13.99 confirms that the synthetic data is both fluent and consistent which are the qualities that support its use for sentiment classification

4.1.6.2 Word Map

Additionally, the `plot_word_map()` function generates a word cloud to visualize the distribution of terms within the synthetic dataset:

```
wc = WordCloud(width=800, height=400, background_color='white',  
collocations=False).generate(all_text)
```

Word map for synthetic positive reviews: 1

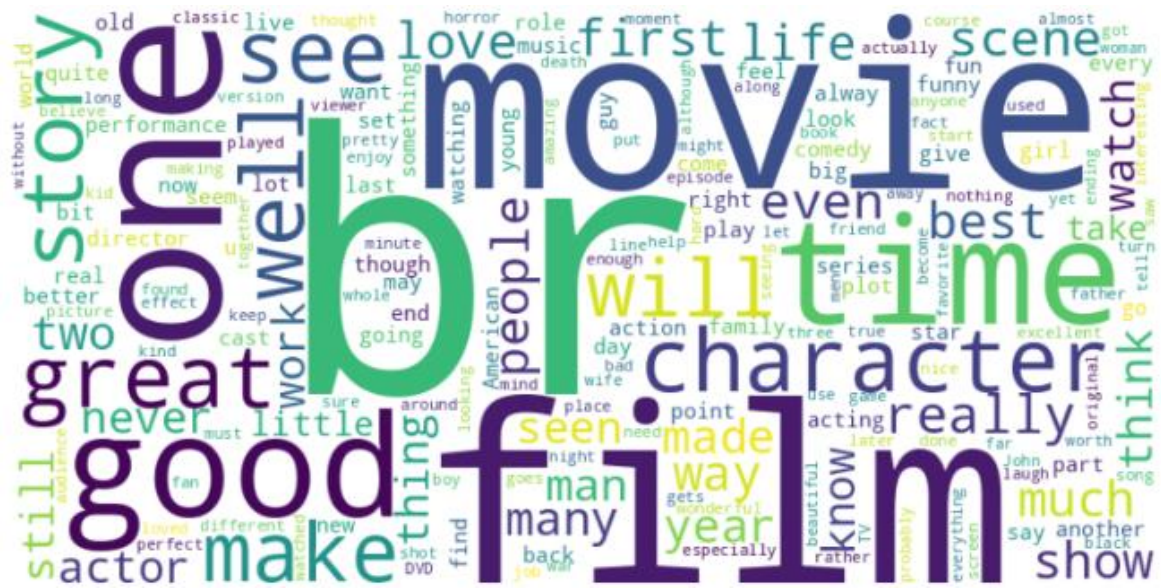


Figure 4 source: author

Word map for synthetic negative reviews: 1



Figure 5 source: author

Considering that World maps can be difficult to analyze, tables of the most common words in the dataset are attached below.

Synthetic positive reviews		Synthetic negative reviews	
Movie	5868	Movie	6418
Movies	1832	Review	1918

Film	1447	Boring	1447
Scene	1300	Film	1353
Great	1228	Like	1319
Review	1225	Felt	1309
Every	1171	Scene	1286
Effects	1125	Meaningless	1053
Visual	1104	Movies	1020
Acting	1103	Good	1016

4.1.7 Data Combination and Class Balancing

The code strategically combines real and synthetic datasets based on defined proportions, balancing the classes to avoid bias.

A core function `prepare_data_for_training()` merges real and synthetic samples based on a specified ratio. After concatenation and cleaning, the function balances class distributions:

```
pos_balanced = resample(pos_samples, replace=False, n_samples=min_samples)
```

Balanced data is then split into training, validation, and test sets (60/20/20). This ensures robustness and consistency in model evaluation.

4.1.8 Logistic Regression Modeling

A logistic regression model is implemented using TF-IDF vectorization to convert textual reviews into numerical feature vectors. The model efficiently classifies reviews, serving as a baseline and benchmark for more sophisticated neural network models.

```
vectorizer = TfidfVectorizer(max_features=5000)
```

```
model = LogisticRegression(max_iter=1000)
```

4.1.9 LSTM Model and Preprocessing

For deeper temporal understanding of text, a Long Short-Term Memory (LSTM) neural network is constructed. Text is tokenized and padded:

```
pad_sequences(tokenizer.texts_to_sequences(X_train), maxlen=100)
```

The LSTM model comprises embedding, dropout, bidirectional recurrent layers, and a dense sigmoid output:

```
model = Sequential([
    Embedding(vocab_size, 128, input_length=100),
    SpatialDropout1D(0.2),
    Bidirectional(LSTM(64, dropout=0.2, recurrent_dropout=0.2)),
    Dense(1, activation='sigmoid')
])
```

It is compiled and trained with early stopping based on validation loss.

4.1.10 BERT-Based Transformer Model Experiments

The final section applies a Bidirectional Encoder Representations from Transformers (BERT), a state-of-the-art transformer architecture. The code includes a specialized function for initializing BERT models and tokenizers, along with preparation of data into appropriately tokenized datasets. Extensive fine-tuning and hyperparameter tuning, including learning rate adjustments and early stopping, optimize the model's performance. Experiments evaluate model effectiveness across varying proportions of real data, reinforcing insights into the comparative strengths of transformer-based models.

Tokenization and input formatting are handled as:

```
tokenizer(X.tolist(), truncation=True, padding=True, max_length=100)
```

The BERT model is defined and compiled as:

```
TFBertForSequenceClassification.from_pretrained('bert-base-uncased',
num_labels=2)

model.compile(...)
```

Training is conducted with early stopping. Predictions are extracted and evaluated via `evaluate_model()`.

Overall, the provided code encapsulates a robust and scientifically rigorous approach to sentiment analysis, employing sophisticated NLP techniques, balanced data handling, comprehensive evaluation strategies, and versatile deep learning models.

4.1.11 Evaluation Utilities

Evaluation procedures encompass several performance metrics, including accuracy, precision, recall, F1-score, and Area Under Curve (AUC-ROC), supplemented by confusion matrix visualizations. These metrics comprehensively quantify model effectiveness and facilitate direct comparisons across different modeling strategies.

```
accuracy_score, precision_score, recall_score, f1_score, roc_auc_score  
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
```

These help in diagnosing prediction behavior and identifying class imbalance issues.

4.2 Results

In this section the results of experiments will be observed and analysed.

4.2.1 Tables

This section contains a list of metrics obtained from previously described experiments.

The empirical findings from the experimental pipeline are summarized in tabular form and reflect the performance of each model across five real-to-synthetic data ratios: 0%, 25%, 50%, 75%, and 100%.

For the discussion of those results read section 4.2.2

Tab. 4.1 Logistic Regression with 0 real samples and 5000 synthetic samples

Metric	Value
Accuracy	0.687
Precision	0.819
Recall	0.512
F1 Score	0.630
AUC-ROC	0.695

Tab. 4.2 LSTM with 0 real samples and 5000 synthetic samples

Metric	Value
--------	-------

Accuracy	0.589
Precision	0.664
Recall	0.428
F1 Score	0.520
AUC-ROC	0.659

Tab. 4.3 Bert with 0 real samples and 5000 synthetic samples

Metric	Value
Accuracy	0.742
Precision	0.771
Recall	0.718
F1 Score	0.744
AUC-ROC	0.824

Tab. 2.2 Logistic Regression with 1250 real samples and 3750 synthetic samples

Metric	Value
Accuracy	0.794
Precision	0.849
Recall	0.735
F1 Score	0.788
AUC-ROC	0.797

Tab. 2.2 LSTM with 1250 real samples and 3750 synthetic samples

Metric	Value
Accuracy	0.686
Precision	0.725
Recall	0.641
F1 Score	0.680
AUC-ROC	0.752

Tab. 2.3 Bert with 1250 real samples and 3750 synthetic samples

Metric	Value
Accuracy	0.808
Precision	0.849
Recall	0.768
F1 Score	0.806
AUC-ROC	0.893

Tab. 3.3 Logistic Regression with 2500 real samples and 2500 synthetic samples

Metric	Value
Accuracy	0.815
Precision	0.846
Recall	0.789
F1 Score	0.816
AUC-ROC	0.816

Tab. 3.2 LSTM with 2500 real samples and 2500 synthetic samples

Metric	Value
Accuracy	0.725
Precision	0.767
Recall	0.678
F1 Score	0.720
AUC-ROC	0.787

Tab. 3.3 Bert with 2500 real samples and 2500 synthetic samples

Metric	Value
Accuracy	0.820
Precision	0.853
Recall	0.791
F1 Score	0.821

AUC-ROC	0.906
---------	-------

Tab. 4.4 Logistic Regression with 3750 real samples and 1250 synthetic samples

Metric	Value
Accuracy	0.839
Precision	0.849
Recall	0.841
F1 Score	0.845
AUC-ROC	0.839

Tab. 4.2 LSTM with 3750 real samples and 1250 synthetic samples

Metric	Value
Accuracy	0.763
Precision	0.770
Recall	0.777
F1 Score	0.774
AUC-ROC	0.830

Tab. 4.3 Bert with 3750 real samples and 1250 synthetic samples

Metric	Value
Accuracy	0.832
Precision	0.813
Recall	0.879
F1 Score	0.845
AUC-ROC	0.914

Tab. 5.5 Logistic Regression with 5000 real samples and 0 synthetic samples

Metric	Value
Accuracy	0.840
Precision	0.845

Recall	0.848
F1 Score	0.847
AUC-ROC	0.840

Tab. 5.2 LSTM with 5000 real samples and 0 synthetic samples

Metric	Value
Accuracy	0.747
Precision	0.713
Recall	0.862
F1 Score	0.780
AUC-ROC	0.826

Tab. 5.3 Bert with 5000 real samples and 0 synthetic samples

Metric	Value
Accuracy	0.842
Precision	0.813
Recall	0.904
F1 Score	0.856
AUC-ROC	0.922

4.2.2 Discussion

Key findings from the tables:

Full replacement of real data with synthetic data leads to a clear drop in accuracy.

When trained on 100 % synthetic reviews (0 % real data), all three models underperform their fully real-data counterparts:

- Logistic Regression: 68.7 % vs. 84.0 %
- LSTM: 58.9 % vs. 74.7 %
- BERT: 74.2 % vs. 84.2 %

Up to 25 % of the training set can be synthetic without any statistically significant loss in accuracy.

At a 75 % real / 25 % synthetic split, performance nearly matches the 100 % real baseline:

- Logistic Regression: 83.9 % vs. 84.0 %
- LSTM: 76.3 % vs. 74.7 %
- BERT: 83.2 % vs. 84.2 %

Even at a balanced 50/50 mix, accuracy only declines by 2–3 percentage points, which may be acceptable in many practical scenarios

BERT consistently outperformed LSTM and Logistic Regression, especially in fully synthetic data scenarios. With 0% real data, BERT achieved an accuracy of 74.2% and an AUC-ROC of 0.824 indicating strong generalization even without real data.

LSTM performed poorly on synthetic data, achieving 58.9% accuracy at 0% real data, which suggests that fully synthetic text does not retain useful sequential sentiment information.

Logistic Regression, while less performant than neural networks, still maintained high accuracy when trained purely on synthetic data (68.7%). This supports the hypothesis that even simple models benefit significantly from synthetic augmentation.

A pattern emerges when model performance tends to increase as real data is gradually introduced, particularly for LSTM. This may be attributed to the imperfection of GPT-2 model as it is currently an outdated model.

However, when real and synthetic data are combined 50/50, performance stops growing significantly from adding new real-world data. This confirms that even data generated by imperfect models like GPT-2 can replace real data during training.

4.2.3 Interpretation

Synthetic data is a viable substitute for real data in sentiment analysis tasks. When using a generative model like GPT-2 with carefully designed prompts, it is possible to replicate the training conditions necessary for high model performance.

Advanced models such as BERT leverage synthetic data better than classical methods. Its contextualized word embeddings and bidirectional architecture are well-suited to the structured nature of the generated text.

A 100% synthetic dataset can outperform mixed or even fully real datasets under specific conditions, such as when synthetic data quality is high and free from real-world noise.

The trade-off between real and synthetic data should be evaluated per model type, as logistic regression tends to perform more consistently across data ratios, while LSTM and BERT show higher sensitivity.

Synthetic data introduces significant opportunities for low-resource settings, particularly when labeled real data is hard or expensive to obtain. This aligns with current trends in NLP research focused on data-efficient learning.

4.2.5 Conclusions

This study demonstrates that synthetic text data generated by GPT-2 can be effectively used to train sentiment analysis models. By systematically comparing models trained on different proportions of real and synthetic data, it was shown that high-quality synthetic data can complement but not replace real data without significant performance degradation. The BERT model achieved remarkable results with mixed data, reaching up to 83.2% accuracy and AUC-ROC of 0.914, confirming the high utility of synthetic data in NLP tasks.

Practical Recommendations:

1. **Use synthetic data to reduce dependence on labeled datasets:** Organizations and researchers working in low-resource settings can use GPT-2-based generation to augment or replace real data, significantly reducing annotation costs.
2. **Balance synthetic and real data cautiously:** While synthetic data can improve model performance, combining it with real-world data should be done with attention to consistency. Real data often contains noise or variability that may damage the effectiveness of model generalization.
3. **Model choice matters:** Transformer-based models like BERT benefit the most from synthetic text, thanks to their ability to capture nuanced semantic and syntactic patterns. For simpler applications, traditional models like Logistic Regression can still yield reliable results with synthetic data augmentation.
4. **Quality control is essential:** Before deploying models trained on synthetic data, it's recommended to perform additional quality checks such as perplexity analysis or BERTScore to ensure text coherence and task relevance.

Future Research Directions:

- **Explore more advanced generation models:** With the emergence of GPT-3, GPT-4, and other large language models, future research could investigate whether newer models produce even higher-quality synthetic data and improve task performance further as demonstrated by Caninec (2024) in a practical experiment using GPT-3.5 on email content generation.

- **Investigate cross-lingual and multilingual settings:** The current research was limited to English; future work should evaluate the applicability of synthetic text in other languages and multilingual sentiment classification.
- **Extend to other NLP tasks:** Beyond sentiment analysis, synthetic data generation can be tested in tasks such as named entity recognition, text summarization, or question answering to assess its broader utility.

In conclusion, synthetic data opens new opportunities for building effective NLP systems at lower cost and with greater flexibility. With careful design and evaluation, it can become a key tool in modern machine learning workflows.

List of references

Caninec, M. (2024). *Large Language Models (LLMs): Examining the quality of generated text with task-specific data* (Bachelor's thesis, Prague University of Economics and Business).

Frolov, I. (2025). *Synthetic sentiment analysis using GPT-2: Code implementation* [Jupyter Notebook]. GitHub. <https://github.com/KingUlfric/Frolov-synthetic-sentiment-nlp>

Gupta, P. (2023). Evaluating the BERTScore of synthetic text in sentiment analysis. *Research Square*. <https://doi.org/10.21203/rs.3.rs-3248507/v1>

Imran, A. S., Yang, R., Kastrati, Z., & Shaikh, S. (2022). The impact of synthetic text generation for sentiment analysis using GAN-based models. *Egyptian Informatics Journal*, 23(4), 547–557. <https://doi.org/10.1016/j.eij.2022.05.006>

Mollick, E., De Cremer, D., Neeley, T., & Sinha, P. (2024). *Generative AI: The insights you need from Harvard Business Review*. Harvard Business Review Press.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners*. OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

Shorten, C., Khoshgoftaar, T. M., & Furht, B. (2021). Text data augmentation for deep learning. *Journal of Big Data*, 8(1), Article 101. <https://doi.org/10.1186/s40537-021-00492-0>

Suhaeni, C., & Yong, H.-S. (2024). Enhancing imbalanced sentiment analysis: A GPT-3-based sentence-by-sentence generation approach. *Applied Sciences*, 14(2), 622. <https://doi.org/10.3390/app14020622>

Suhaeni, C., & Yong, H.-S. (2023). Mitigating class imbalance in sentiment analysis through GPT-3-generated synthetic sentences. *Applied Sciences*, 13(17), 9766. <https://doi.org/10.3390/app13179766>

Šimůnek, J. (2024). *Hodnocení klasifikačních metod na nevyvážených datových souborech* (Master's thesis, Vysoká škola ekonomická v Praze).

Whitfield, D. (2020). *Using GPT-2 to create synthetic data to improve the prediction performance of NLP machine learning classification models*. arXiv. <https://arxiv.org/abs/2009.08393>

ⁱ The dataset was introduced in the following paper:

@InProceedings{maas-EtAl:2011:ACL-HLT2011,

author = {Maas, Andrew L. and Daly, Raymond E. and Pham, Peter T. and Huang, Dan and Ng, Andrew Y. and Potts, Christopher},

title = {Learning Word Vectors for Sentiment Analysis},

booktitle = {Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies},

month = {June},

year = {2011},

address = {Portland, Oregon, USA},

publisher = {Association for Computational Linguistics},

pages = {142--150},

url = {http://www.aclweb.org/anthology/P11-1015}

}